Dataset Code:

```
GNU nano 7.2                                    generate_data.py *
import random
N = 10000
with open('tasks.txt', 'w') as f:
    for i in range(1, N+1):
        # Cost proportional to index (heterogeneous workload)
        cost = i * random.randint(1, 10)
        f.write(f'TASK{i} {cost}\n')

^G Help      ^O Write Out   ^W Where Is   ^K Cut       ^T Execute   ^C Location   M-U Undo      M-A Set Mark
^X Exit      ^R Read File   ^\ Replace    ^U Paste     ^J Justify   ^/ Go To Line M-E Redo      M-6 Copy
```

Serial code:

```
GNU nano 7.2                                    serial_process.c *
#include <stdio.h>
#include <time.h>
#define N 10000
long long tasks[N];
void execute_task(int i) {
    volatile double result = 0;
    for (long long j = 0; j < tasks[i]; j++)
        result += j * 0.001;
}
int main() {
    for (int i = 0; i < N; i++) tasks[i] = (i+1) * 100;
    clock_t start = clock();
    for (int i = 0; i < N; i++) execute_task(i);
    double t = (double)(clock()-start)/CLOCKS_PER_SEC;
    printf("Serial time: %.4f sec\n", t);
    return 0;
}

^G Help      ^O Write Out   ^W Where Is   ^K Cut       ^T Execute   ^C Location   M-U Undo      M-A Set Mark
^X Exit      ^R Read File   ^\ Replace    ^U Paste     ^J Justify   ^/ Go To Line M-E Redo      M-6 Copy
```

Serial code output:

```
vyshnavi_vugge@LAPTOP-6JE
Microsoft Windows [Version 10.0.26200.8117]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Vyshnavi>wsl
vyshnavi_vugge@LAPTOP-6JB76F7R:/mnt/c/Users/Vyshnavi$ cd ~
vyshnavi_vugge@LAPTOP-6JB76F7R:~$ mkdir HPC_TaskScheduling_Project
vyshnavi_vugge@LAPTOP-6JB76F7R:~$ cd HPC_TaskScheduling_Project
vyshnavi_vugge@LAPTOP-6JB76F7R:~/HPC_TaskScheduling_Project$ nano generate_data.py
vyshnavi_vugge@LAPTOP-6JB76F7R:~/HPC_TaskScheduling_Project$ python3 generate_data.py
vyshnavi_vugge@LAPTOP-6JB76F7R:~/HPC_TaskScheduling_Project$ nano serial_process.c
vyshnavi_vugge@LAPTOP-6JB76F7R:~/HPC_TaskScheduling_Project$ gcc serial_process.c -o serial_process
vyshnavi_vugge@LAPTOP-6JB76F7R:~/HPC_TaskScheduling_Project$ ./serial_process
Serial time: 18.4585 sec
vyshnavi_vugge@LAPTOP-6JB76F7R:~/HPC_TaskScheduling_Project$ nano parallel_process.c
vyshnavi_vugge@LAPTOP-6JB76F7R:~/HPC_TaskScheduling_Project$
```

Parallel code:

```
vyshnavi_vugge@LAPTOP-6JE
  GNU nano 7.2                                   parallel_process.c *

#include <stdio.h>
#include <omp.h>
#include <stdlib.h>
#include <time.h>

#define N 10000
#define CHUNK 50

long long tasks[N];

void execute_task(int i) {
    volatile double r = 0;
    for (long long j = 0; j < tasks[i]; j++) {
        r += j * 0.001;
    }
}

// We will use ONLY one parallel run (best: guided)
double run_parallel() {

    double t = omp_get_wtime();

    #pragma omp parallel for schedule(guided, CHUNK)
    for (int i = 0; i < N; i++) {
        execute_task(i);
    }

    return omp_get_wtime() - t;
}

int main() {

                                          [ Read 49 lines ]
^G Help      ^O Write Out   ^W Where Is    ^K Cut       ^T Execute   ^C Location   M-U Undo     M-A Set Mark
^X Exit      ^R Read File   ^\ Replace     ^U Paste     ^J Justify   ^/ Go To Line M-E Redo     M-6 Copy
```

```
vyshnavi_vugge@LAPTOP-6JE
  GNU nano 7.2                                   parallel_process.c *
// We will use ONLY one parallel run (best: guided)
double run_parallel() {

    double t = omp_get_wtime();

    #pragma omp parallel for schedule(guided, CHUNK)
    for (int i = 0; i < N; i++) {
        execute_task(i);
    }

    return omp_get_wtime() - t;
}

int main() {

    omp_set_num_threads(8);   // match your expected output

    srand(time(NULL));        // random workload

    for (int i = 0; i < N; i++) {
        tasks[i] = (rand() % 1000 + 1) * 1000;
    }

    double parallel_time = run_parallel();

    // ✅ Clean HPC-style output
    printf("Parallel Time : %.4f seconds\n", parallel_time);
    printf("CPUs Used     : %d\n", omp_get_max_threads());
    printf("Records       : %d\n", N);

    return 0;
}

^G Help      ^O Write Out   ^W Where Is    ^K Cut       ^T Execute   ^C Location   M-U Undo     M-A Set Mark
^X Exit      ^R Read File   ^\ Replace     ^U Paste     ^J Justify   ^/ Go To Line M-E Redo     M-6 Copy
```

Parallel code output:

```
vyshnavi_vugge@LAPTOP-6JB76F7R:~/HPC_TaskScheduling_Project$ nano parallel_process.c
vyshnavi_vugge@LAPTOP-6JB76F7R:~/HPC_TaskScheduling_Project$  gcc parallel_process.c -fopenmp -o parallel_process
vyshnavi_vugge@LAPTOP-6JB76F7R:~/HPC_TaskScheduling_Project$ ./parallel_process
Parallel Time : 3.2666 seconds
CPUs Used     : 8
Records       : 10000
vyshnavi_vugge@LAPTOP-6JB76F7R:~/HPC_TaskScheduling_Project$
```

Comparision Code:

```
GNU nano 7.2                                        compare.c *
#include <stdio.h>
#include <time.h>
#include <omp.h>

#define N 10000

long long tasks[N];

// Work function
void execute_task(int i) {
    volatile double result = 0;
    for (long long j = 0; j < tasks[i]; j++) {
        result += j * 0.001;
    }
}

int main() {

    // Initialize workload
    for (int i = 0; i < N; i++) {
        tasks[i] = (i + 1) * 100;
    }

    // ---------------- SERIAL ----------------
    clock_t start_serial = clock();

    for (int i = 0; i < N; i++) {
        execute_task(i);
    }

    double serial_time = (double)(clock() - start_serial) / CLOCKS_PER_SEC;

    // ---------------- PARALLEL ----------------

^G Help      ^O Write Out   ^W Where Is   ^K Cut       ^T Execute   ^C Location   M-U Undo      M-A Set Mark
^X Exit      ^R Read File   ^\ Replace    ^U Paste     ^J Justify   ^/ Go To Line  M-E Redo      M-6 Copy
```

```
GNU nano 7.2                                                    compare.c *
        tasks[i] = (i + 1) * 100;
    }

    // ---------------- SERIAL ----------------
    clock_t start_serial = clock();

    for (int i = 0; i < N; i++) {
        execute_task(i);
    }

    double serial_time = (double)(clock() - start_serial) / CLOCKS_PER_SEC;

    // ---------------- PARALLEL ----------------
    double start_parallel = omp_get_wtime();

    #pragma omp parallel for schedule(dynamic)
    for (int i = 0; i < N; i++) {
        execute_task(i);
    }

    double parallel_time = omp_get_wtime() - start_parallel;

    // ---------------- RESULTS ----------------
    double speedup = serial_time / parallel_time;

    printf("\n--- Performance Comparison ---\n");
    printf("Serial Time   : %.4f sec\n", serial_time);
    printf("Parallel Time : %.4f sec\n", parallel_time);
    printf("Speedup       : %.2f\n", speedup);

    return 0;
}

^G Help      ^O Write Out   ^W Where Is   ^K Cut       ^T Execute   ^C Location   M-U Undo      M-A Set Mark
^X Exit      ^R Read File   ^\ Replace    ^U Paste     ^J Justify   ^/ Go To Line M-E Redo      M-6 Copy
```

Comparision output:

```
vyshnavi_vugge@LAPTOP-6JB76F7R:~/HPC_TaskScheduling_Project$ nano parallel_process.c
vyshnavi_vugge@LAPTOP-6JB76F7R:~/HPC_TaskScheduling_Project$ nano compare.c
vyshnavi_vugge@LAPTOP-6JB76F7R:~/HPC_TaskScheduling_Project$ gcc compare.c -fopenmp -o campare
vyshnavi_vugge@LAPTOP-6JB76F7R:~/HPC_TaskScheduling_Project$ ./campare

--- Performance Comparison ---
Serial Time   : 19.4204 sec
Parallel Time : 2.7366 sec
Speedup       : 7.10
vyshnavi_vugge@LAPTOP-6JB76F7R:~/HPC_TaskScheduling_Project$
```